

# 10주차\_5~6장(~TF-IDF)\_자연어처리와 컴퓨터비전 심층학습

## 5. 토큰화

- 자연어 처리: 컴퓨터가 인간의 언어를 이해하고 처리하도록 하는 것
- 해결해야할 문제:
  - 모호성: 맥락에 따라 여러 의미를 가지므로 모호함
  - 가변성: 사투리, 강세, 신조어, 작문 스타일로 인해 매우 가변적임
  - 구조: 문장의 구조와 문법적 요소를 이해할 수 있어야 함.
- **토큰화(Tokenization)**: **말뭉치(Corpus)**를 **토큰(Token)**으로 나누는 것
  - 말뭉치: 자연어 모델에서 사용하는 대규모의 자연어 (텍스트 데이터)
  - 토큰: 말뭉치보다 더 작은 단위 (ex. 개별 단어, 문장 부호 등)
  - 토큰화를 위해 **토큰나이저(Tokenizer)**를 사용한다.
- 토큰나이저 구축하는 방법:
  - 공백 분할: 텍스트를 공백 단위로 분리해 토큰화
  - 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트 분할
  - 어휘 사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용
    - OOV(out of vocab) 문제: 어휘 사전에 없는 단어가 존재하는 문제
    - 차원의 저주: 모든 토큰을 벡터화하면 차원이 너무 커지고 희소해짐
    - 토큰들의 순서 관계 표현 X
  - 머신러닝 활용: 데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝 적용

### 1. 단어 및 글자 토큰화

#### [ 단어 토큰화 ]

- 텍스트 데이터를 의미 있는 단위인 단어로 분리
- 특정 구분자(띄어쓰기, 문장 부호, 대소문자 등)를 활용
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 사용
- `.split()` 함수 사용
- 단점: 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류 등에 취약

## [ 글자 토큰화 ]

- 글자 단위로 문장을 나눔
- 시퀀스 예측 작업에서 사용
- `list()` : 리스트로 변환, 공백도 토큰으로 나눔
- 한글의 경우, **자소 단위 토큰화** 수행 (자음, 모음을 나눔)
  1. `jamo.h2j()` : 완성형으로 입력된 한글 문자열을 조합형 한글 자모로 변환
  2. `jamo.j2hcj()` : 조합형 한글 문자열을 자소 단위로 나눠 반환
    - 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현
    - 완성형: 조합된 글자 자체에 값을 부여해 인코딩
- 장점:
  - 비교적 작은 단어 사전을 구축할 수 있음  
→ 컴퓨터 자원 덜 사용, 각 단어를 더 자주 학습할 수 있음
  - 접사와 문장 부호의 의미를 학습할 수 있다.
- 단점:
  - 토큰 조합해 의미 도출하는 방식 → 다의어, 동음이의어 구별 어려움
  - 모델 입력 시퀀스의 길이가 길어질수록 연산량이 증가

## 2. 형태소 토큰화

- 언어의 문법과 구조를 고려해 단어를 분리
- 각 단어가 띄어쓰기로 구분되지 않는 교착어(ex.한국어)인 언어에서 중요하게 수행
- 형태소: 의미를 가지고 있는 최소의 단위

- 자립 형태소: 스스로 의미를 가짐 → 단어(명사, 동사, 형용사 등)를 이루는 기본 단위
- 의존 형태소: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용 → 조사, 어미, 접두사, 접미사 등

## [ 형태소 어휘 사전 ]

- 각 단어의 형태소 정보가 저장된 사전
- 각 형태소가 어떤 품사에 속하는지 + 품사의 뜻을 함께 제공
- 품사 태깅 (POS Tagging): 텍스트 데이터를 형태소 분석하여 품사를 태깅하는 작업

## [ KoNLPy ]

- 자바 언어 기반의 한국어 형태소 분석기 → 자바 개발 키트 설치 필요
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
- Okt (SNS 텍스트 데이터 기반), 꼬꼬마 (국립국어원 세종 말뭉치 기반), 코모란, 한나눔, 메캅 등의 다양한 형태소 분석기 지원
- 동일한 메서드라도 형태소 분석기의 특징에 따라 다른 결과 나오기 때문에 적절한 형태소 분석기 사용해야 함.

```
# Okt 토큰화
from konlpy.tag import Okt
okt=Okt()
sentence=' ___ '
nouns=okt.nouns(sentence)      # 명사 추출
phrases=okt.phrases(sentence)  # 구 추출
morphs=okt.morphs(sentence)    # 형태소 추출
pos=okt.pos(sentence)          # 품사 태깅
```

```
# 꼬꼬마 토큰화
from konlpy.tag import Kkma
kkma=Kkma()
sentence=' ___ '
nouns=kkma.nouns(sentence)      # 명사 추출
phrases=kkma.sentences(sentence) # 문장 추출
```

```
morphs=kkma.morphs(sentence)      # 형태소 추출
pos=kkma.pos(sentence)             # 품사 태깅
```

## [ NLTK ]

- 다양한 언어(주로 영어)의 자연어 처리를 위한 데이터와 모델 제공
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등의 기능 제공
- 토큰화/품사 태깅 위해서는 Punkt 모델이나 Averaged Perceptron Tagger 모델을 다운로드해야 함

```
pip install nltk      # 라이브러리 설치
import nltk

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

from nltk import tokenize
from nltk import tag
sentence=' ___ '
word_tokens=tokenize.word_tokenize(sentence) #공백 기준 단어 분리
sent_tokens=tokenize.sent_tokenize(sentence) #구두점 기준 문장 분리
pos=tag.pos_tag(word_tokens)  # 품사 태깅
```

## [ spaCy ]

- 사이언 기반 오픈 소스 라이브러리
- 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리
- NLTK 모델보다 더 크고, 복잡하고, 더 많은 리소스 요구
- GPU 가속, 24개 이상 언어로 사전학습된 모델 제공

```
pip install spacy
python -m spacy download en_core_web_sm  #영어로 사전학습된 모델 설치
```

```
import spacy
nlp=spacy.load('en_core_web_sm') # 모델 불러오기 함수
sentence=' __ '
doc=nlp(sentence) # 토큰 별로 pos_, tag_, text 등의 속성 저장
```

### 3. 하위 단어 토큰화

- 기존 형태소 분석기: 외래어, 띄어쓰기 오류, 오타자 등이 있는 문장을 토큰화하기 어려움. 즉 모르는 단어를 적절한 단위로 나누는 것에 취약
- 하위 단어 토큰화: 단어를 하위 단어(subword)의 조합으로 나눠 토큰화하는 방법
  - 단어의 길이를 줄여 처리 속도가 빨라짐
  - OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제 완화
- 바이트 페어 인코딩, 워드피스, 유니그램 모델 등이 있다.

#### [ 바이트 페어 인코딩 ]

- 바이트 페어 인코딩 (BPE, 다이그램 코딩): 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘
  - 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 **조합 탐지와 부호화를 반복**
  - 가장 자주 등장하는 단어를 하나의 토큰으로 토큰화 → 덜 등장하는 단어는 여러 토큰의 조합으로 표현
- 대량의 말뭉치를 사용해 토크나이저를 학습한다면 이미 구현된 토크나이저 라이브러리를 사용함
  - 코포라
    - 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리
    - `Korpora.load()` : 말뭉치 데이터 다운로드
    - `get_all_texts()` : 본문 데이터 한 번에 불러옴
  - 센텐스피스 라이브러리
    - 구글에서 개발한 오픈소스 하위 단어 토크나이저 라이브러리

- `SentencePieceTrainer.Train(...)`: 토크나이저 모델 학습
  - `petition_bpe.model`: 학습된 토크나이저가 저장된 파일
  - `petition_bpe.vocab`: 어휘 사전이 저장된 파일
- `SentencePieceProcessor` 클래스로 학습된 모델 불러옴

```
# 학습 데이터셋 생성
from Korpora import Korpora
corpus=Korpora.load('korean_petitions') #청와대 청원 데이터
dataset=corpus.train #train 속성으로 학습 데이터 가져옴
petition=dataset[0]
print(petition.begin)      # 청원 시작일
print(petition.end)        # 청원 종료일
print(petition.num_agree)   # 청원 동의 수
print(petition.category)   # 청원 범주
print(petition.title)      # 청원 제목
print(petition.text)       # 청원 본문
petitions=corpus.get_all_texts() # 본문 데이터 한번에

# 토크나이저 모델 학습
from sentencepiece import SentencePieceTrainer
SentencePieceTrainer.Train(
    )

# 토큰화
from sentencepiece import SentencePieceProcessor
tokenizer=SentencePieceProcessor()
tokenizer.load('petition_bpe.model')
tokenizer_sentence=tokenizer.encode_as_pieces(sentence) #
문장 토큰화
encoded_sentence=tokenizer.as_ids(sentence) # 토큰을 정수로 인
코딩
decode_ids=tokenizer.decode_ids(encoded_sentence) # 정수를
문자열 데이터로 변환
decode_pieces=tokenizer.decode_pieces(encoded_sentence)

# 어휘 사전 불러오기
vocab={idx:tokenizer.id_to_piece(idx) for idx in range(toke
```

```
nizer.get_piece_size())}
print(list(vocab.items())[:5])
print('vocab size:', len(vocab))
```

## [ 워드피스 알고리즘 ]

- BPE와 유사하지만, 빈도 기반이 아닌 확률 기반으로 하위 단어를 병합
- 하위 단어 간 함께 나타날 확률 계산

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x, y)}{f(x) \cdot f(y)}$$

- f는 빈도, x와 y는 병합하려는 하위 단어
- x와 y를 병합하는 것이 적절한지 판단하는 점수
- 가장 높은 확률을 가진 하위 단어 쌍을 선택
- **토큰라이저스 라이브러리**
  - 정규화: 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점/특수문자 처리 등
  - 사전 토큰화: 토큰화 전에 작은 단위로 나눔 (ex. 단어)

## 6. 임베딩

- **텍스트 벡터화**: 텍스트를 숫자로 변환하는 과정
  - 원-핫 인코딩: 각 단어를 고유한 색인 값으로 매핑 후 해당 색인 위치를 1로 표시하고 나머지는 모두 0으로 표시
  - 빈도 벡터화: 해당 단어의 빈도를 벡터로 표시
 → 벡터의 **희소성이 크다** ⇒ **워드 임베딩 기법 사용**
- **워드 임베딩 (Word Embedding)**: 단어를 고정된 길이의 실수 벡터로 표현
  - 다른 단어와의 상대적 위치로 단어 간의 관계를 추론
  - ex) 워드 투 벡터(Word2Vec), 패스트 텍스트(fastText)

- 다의어나 문맥 정보 다루기 어려움 → **동적 임베딩** 기법 사용

## 1. 언어 모델

- **언어 모델**: 입력된 문장 다음으로 각 문장이 나올 확률을 계산하는 모델
  - 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성을 예측
  - 완성된 문장 전체의 확률 계산은 어려움 → 자기회귀 언어 모델 고안

### [ 자기 회귀 언어 모델 ]

- **자기 회귀 언어 모델**: 이전 단어들의 시퀀스를 바탕으로 다음 단어의 확률을 계산해 다음에 올 단어를 예측

수식 6.1 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_i | w_1, w_2, \dots, w_{i-1}) = P(w_1)P(w_2 | w_1) \dots P(w_i | w_1, w_2, \dots, w_{i-1})$$

→ 이전 단어들의 조건부 확률로 다음에 등장하는 단어의 확률을 계산

- 이전에 등장한 모든 토큰의 정보를 활용해 다음 토큰을 예측

### [ 통계적 언어 모델 ]

- **통계적 언어 모델**: 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악, 다음에 등장할 단어의 확률을 예측
  - 마르코프 체인을 이용해 구현됨
- **마르코프 체인**: 빈도 기반의 조건부 확률 모델
  - 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측
  - 단어의 순서와 빈도수를 기반으로 확률을 계산
  - 문맥 이해 부족 문제 있음
  - **데이터 희소성 문제**: 관측한 적 없는 데이터를 예측하지 못함
    - 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를



## 생성

- 최근 자연어 처리는 언어 모델을 활용해 텍스트 데이터를 사전학습함.
  - GPT (Generative Pre-trained Transformer): 이전 단어들을 바탕으로 다음 단어를 순차적으로 예측
  - BERT(Bidirectional Encoder Representations from Transformers): 문장 속 일부 단어를 가리고, 주변 문맥을 이용해 빈칸을 예측

## 2. N-gram

- 가장 기초적인 통계적 언어 모델
- N개의 연속된 단어 시퀀스를 하나의 단위로 취급
  - N=1: 유니그램
  - N=2: 바이그램
  - N=3: 트라이그램
  - N≥4: N-gram
- N-1개의 토큰만 고려해 확률 계산
- t번째 토큰의 조건부 확률 계산 수식:

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- t: 예측하려는 단어
  - t-1~t-N+1: 예측에 사용되는 이전 단어들
- 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석할 때 효과적
- 관용적 표현 분석에 사용

```
# 파이썬 코드로 구현
def ngrams(sentence, n):
    words=sentence.split()
    ngrams=zip(*[words[i:] for i in range(n)])
    return list(ngrams)
```

```

unigram=ngrams(sentence,1)
bigram=ngrams(sentence,2)
trigram=ngrams(sentence,3)

# NLTK 라이브러리에서 지원하는 N-gram 함수
import nltk

unigram=nltk.ngrams(sentence.split(),1)
bigram=nltk.ngrams(sentence.split(),2)
trigram=nltk.ngrams(sentence.split(),3)

```

### 3. TF-IDF

- **TF-IDF** (Term Frequency-Inverse Document Frequency): 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치  
→ BoW에 가중치를 부여하는 방법
- **BoW** (Bag-of-Words): 텍스트에 등장하는 단어의 빈도를 기록하는 것
  - 모든 단어는 동일한 가중치를 가짐  
→ I don't like this 에서 don't라는 단어가 중요하지만 자주 등장하지 않으므로 무시될 수 있음
- **단어 빈도** (Term Frequency, TF): 특정 단어의 빈도수
  - 단어의 상대적인 중요도 측정
  - TF 높음 = 특정 문서에서 중요할 수도 있으나 전문 용어나 관용어일 수 있다
  - 문서가 길어질수록 해당 단어의 TF값도 높아질 수 있음
- **문서 빈도** (Document Frequency, DF): 특정 단어가 몇 개의 문서에서 나타나는지
  - DF 높음 = 일반적으로 널리 사용, 중요도 낮음
  - DF 낮음 = 특정 문맥에서만 사용, 중요도 높음
- **역문서 빈도** (Inverse Document Frequency, IDF): 문서 내에서 특정 단어가 얼마나 중요한지 나타냄

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- 역문서 빈도 낮음 = 여러 문서에 자주 등장
- 분모에 1을 더해 분모가 0이 되는걸 방지
- 로그를 씌워 값이 너무 커지는걸 방지
- **TF-IDF** : 문서 빈도와 역문서 빈도를 곱한 값

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- TF-IDF 큼 = 문서 내에서 단어가 자주 등장하지만 전체 문서 중에서는 적게 등장
- 관사, 관용어 등의 가중치 낮아짐
- 사이킷런은 텍스트 데이터를 쉽게 TF-IDF 형태로 변환할 수 있는 `TfidfVectorizer` 클래스 제공

```
# sklearn 이용해 TF-IDF 계산
tfidf_vectorizer = sklearn.feature_extraction.text.TfidfVectorizer(
    input="content",
    encoding="utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1, 1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
    smooth_idf=True,
)
```